

深圳大学实验报告

课程名称： 算法设计与分析

实验名称： 实验六 用最大流算法解决问题

学院： 计算机与软件学院 专业： 计算机科学与技术

报告人： 黄家成 学号： 2024150286 班级： 国际班

同组人： 黄瑜

指导教师： 吴定明

实验时间：

实验报告提交时间：

一、实验原理与流网络的构造原理

棒球淘汰问题的核心在于：判断某支球队 x 是否在理论上已经失去了夺冠的可能性。

假设球队 x 赢下了所有剩余的比赛，其最终能够达到的最大胜场数为 $W = w_x + r_x$ 。为了让球队 x 有机会夺冠，必须存在一种后续比赛的胜负分配方式，使得其他所有球队的最终胜场数都不会超过 W 。

流网络的构造原理

我们将“剩余比赛的胜负分配”转化为一个网络流模型。在这个网络中，“流”代表着“比赛的胜利场次”。网络包括源点 S 、汇点 T 、比赛节点和球队节点。对于要判断的球队 x ，我们构造如下网络（不包含球队 x 本身）：

- **源点 S 与比赛节点：** 对于任意两支球队 i 和 j ($i \neq x, j \neq x$ 且 $i < j$)，如果在它们之间还有 r_{ij} 场比赛未打，则创建一个比赛节点 G_{ij} 。
 - 添加一条从源点 S 到 G_{ij} 的边，容量为 r_{ij} 。这表示这两支球队之间还有 r_{ij} 场胜利需要被分配。
- **比赛节点与球队节点：** 对于每个比赛节点 G_{ij} ，胜利要么归属于球队 i ，要么归属于球队 j 。
 - 添加一条从 G_{ij} 到球队节点 T_i 的边，容量为 ∞ （无穷大）。
 - 添加一条从 G_{ij} 到球队节点 T_j 的边，容量为 ∞ 。
- **球队节点与汇点 T ：** 为了保证任何球队的最终胜场数不超过 x 的最大理论胜场 W ，球队 i 最多还能赢 $W - w_i$ 场比赛。
 - 添加一条从球队节点 T_i 到汇点 T 的边，容量为 $W - w_i$ 。如果 $W - w_i < 0$ ，说明球队 i 目前的胜场已经超过了 x 能达到的上限，球队 x 显然已被淘汰（平凡淘汰）。

二、最大流解决该问题的正确性及关键定理证明

1.为什么最大流能解决这个问题？

在该流网络中，从源点 S 流出的总容量为所有剩余相关比赛的总数： $C_{total} = \sum(i < j) r_{ij}$ 。

如果我们能找到一个流，其大小等于 C_{total} ，这意味着所有的剩余比赛都找到了赢家，同时受限于连接汇点 T 的边容量，没有任何一支球队的最终总胜场超过 W 。此时，球队 x 未被淘汰。反之，如果最大流严格小于 C_{total} ，则必然发生“容量瓶颈”，即某些比赛的胜利无论怎么分配，都会导致至少一支球队的胜场数超过 W ，从而断定球队 x 被淘汰。

2.关键定理及其证明

定理： 球队 x 被淘汰，当且仅当存在一个球队集合 R (R 属于除去 x 之外的所有球队集合)，使得该集合内球队的当前总胜场加上它们之间剩余比赛的总场数，平均分配后严格大于球队 x 的最大可能胜场 W 。即： $[\sum(i \in R) w_i + \sum(i, j \in R \text{ 且 } i < j) r_{ij}] / |R| > W$

证明与最小割的联系： 根据最大流最小割定理，如果最大流小于从 S 出发的总容量 C_{total} ，则必然存在一个容量小于 C_{total} 的 S - T 割 (A, B) ，其中源点

$S \in A$, 汇点 $T \in B$ 。

- a. 在这个最小割中, 源点 S 属于集合 A 。令 R 为集合 A 中包含的球队节点的集合。
- b. 如果某个球队 $i \in R$, 那么边 (T_i, T) 必定在割集中, 其对割容量的贡献为 $W - w_i$ 。
- c. 如果球队 $i \in R$ 且 $j \in R$, 则比赛节点 G_{ij} 也必须在 A 中 (否则会有容量为 ∞ 的边穿过割, 矛盾)。此时, 边 (S, G_{ij}) 不在割集中。
- d. 如果某场比赛涉及到 R 之外的球队 (即不在 A 中), 则边 (S, G_{ij}) 在割集中, 贡献为 r_{ij} 。
- e. 最小割的容量 $C(A, B)$ 可以表示为: $C(A, B) = \sum(i, j \notin R) r_{ij} + \sum(i \in R) (W - w_i)$ 。
- f. 因为最大流 = 最小割 $< C_{total}$, 即: $\sum(i \notin R \text{ 或 } j \notin R) r_{ij} + \sum(i \in R) (W - w_i) < \sum(\text{所有 } i < j) r_{ij}$
- g. 两边消去非 R 集合内的比赛场数, 得到: $\sum(i \in R) (W - w_i) < \sum(i, j \in R \text{ 且 } i < j) r_{ij} |R| * W - \sum(i \in R) w_i < \sum(i, j \in R \text{ 且 } i < j) r_{ij}$
- h. 移项即得定理中的不等式。该定理的作用在于: 它不仅能判断是否淘汰, 还能给出一个“淘汰证明”, 即通过集合 R 直观解释为什么该球队无法夺冠。

三、代码设计思路与关联分析

本实验通过代码实现网络流图的构建与最大流求解。为了使逻辑更加直观, 代码采用了基于广度优先搜索寻找增广路的方法, 并使用二维数组 (邻接矩阵) 来表示残量网络

1. 代码设计思路与映射关系

平凡淘汰检测: 在构建复杂的流网络之前, 代码首先计算目标球队 x 若赢得所有剩余比赛后的最大理论胜场数 $\text{max_possible_wins} = \text{wins}[x] + \text{left}[x]$ 。如果遍历发现任意一支其他球队目前的胜场 $\text{wins}[i]$ 已经大于这个最大值, 则说明球队 x 已经被平凡淘汰, 直接返回结果, 无需建图。

图节点的映射: 代码中需要将逻辑节点转化为一维数组的索引。

源点: 设为索引 0。

比赛节点: 索引从 1 开始, 共有组合数 $C(N-1, 2)$ 个相关比赛节点。

球队节点: 紧随比赛节点之后, 分配 $N-1$ 个索引。

汇点: 设为总节点数减 1。

网络流动的模拟: 核心函数 `getMaxFlow` 通过不断调用 `bfs` 在残量网络 `graph` 中寻找从源点到汇点的连通路径。一旦找到一条路径, 就计算这条路径上的“最小剩余容量”, 并在整条路径上减去该容量 (正向边推流), 同时在反向边上加上该容量 (允许退流), 直到再也找不到连通路径为止。

2. 核心伪代码设计

Algorithm: `CheckElimination(Team x)`

Input: 目标球队 x , 所有球队当前胜场 `wins[]`, 剩余场次 `left[]`, 对阵矩阵 `g[][]`

Output: 球队 x 是否被淘汰

```

max_wins = wins[x] + left[x]
FOR 每一支其他球队 i (i ≠ x):
    IF wins[i] > max_wins:
        PRINT "球队 x 被平凡淘汰"
    RETURN
初始化残量网络 graph 为全 0 矩阵
total_remaining_games = 0

FOR 每一对其他球队 (i, j) 且 i < j:
    IF 球队 i 和 j 之间还有剩余比赛 g[i][j] > 0:
        创建比赛节点 game_node
        graph[源点][game_node] = g[i][j]
        total_remaining_games += g[i][j]
        graph[game_node][球队 i 节点] = 无穷大
        graph[game_node][球队 j 节点] = 无穷大

FOR 每一支其他球队 i (i ≠ x):
    graph[球队 i 节点][汇点] = max_wins - wins[i]
max_flow = GetMaxFlow(graph, 源点, 汇点)
IF max_flow < total_remaining_games:
    PRINT "球队 x 被淘汰"
ELSE:
    PRINT "球队 x 未被淘汰"

```

Algorithm: GetMaxFlow(graph, source, sink)

```

max_flow = 0
WHILE 使用 BFS 能在 graph 中找到从 source 到 sink 的路径 path:
    path_flow = path 上的最小边容量
    FOR path 上的每一条边 (u, v):
        graph[u][v] -= path_flow // 正向边减流
        graph[v][u] += path_flow // 反向边加流
    max_flow += path_flow
RETURN max_flow

```

四、 算法复杂度分析

本实验采用了基于邻接矩阵和 BFS 增广路的最大流算法，针对棒球淘汰问题的特殊网络结构，其复杂度分析如下：

1. 时间复杂度

在流网络中，设总节点数为 V ，总边数为 E 。

节点数量估算：源点和汇点共 2 个；对于 N 支球队，除掉正在计算的球队后，最多有 $(N-1)(N-2)/2$ 个比赛节点，以及 $N-1$ 个球队节点。因此，节点总数 V 约为 $O(N^2)$ 。

边的数量估算：源点连出 $O(N^2)$ 条边，比赛节点连出 $O(N^2)$ 条边，球队节

点连出 $O(N)$ 条边。因此，边总数 E 约为 $O(N^2)$ 。

最大流计算：使用 Edmonds-Karp 算法思想，寻找增广路的最坏情况需要寻找 $O(V * E)$ 次，每次 BFS 遍历图的时间为 $O(E)$ 。理论上的最坏时间复杂度为 $O(V * E^2)$ 。代入本题参数，时间复杂度约为 $O(N^6)$ 。

实际性能：尽管理论上界较高，但棒球问题的流网络是一个非常浅的二部图结构（源点 $\rightarrow$ 比赛 $\rightarrow$ 球队 $\rightarrow$ 汇点，最长路径仅为 3）。因此在实际运行中，BFS 极快就能触达汇点，增广次数远远低于理论上限，算法在处理常规数量的球队时几乎是瞬间完成的。

2. 空间复杂度

代码中为了逻辑清晰，采用了二维数组 `vector<vector<int>> graph` 来表示流网络（邻接矩阵）。

矩阵的大小为 $V * V$ ，因此空间复杂度为 $O(V^2)$ 。由于 $V = O(N^2)$ ，最终的空间复杂度为 $O(N^4)$ 。对于 N 一般不超过几十支球队的体育联赛数据而言，这种空间开销完全在现代计算机的内存承受范围之内。

五、 利用算法求解棒球问题与运行验证

为了验证算法的正确性，我们将实验题目中给出的旧金山纪事报报道的 4 支球队数据作为输入，带入我们设计的最大流程序中进行验证。

1.求解过程说明

输入数据为：

Atlanta（亚特兰大）：83 胜，剩余 8 场，对阵情况 [0, 1, 6, 1]

Philly（费城）：80 胜，剩余 3 场，对阵情况 [1, 0, 0, 2]

New York（纽约）：78 胜，剩余 6 场，对阵情况 [6, 0, 0, 0]

Montreal（蒙特利尔）：77 胜，剩余 3 场，对阵情况 [1, 2, 0, 0]

程序依次对这四支队伍进行判定：

a. 计算 Montreal 时，其最高理论胜场为 $77+3=80$ 场，而 Atlanta 已有 83 胜，触发“平凡淘汰”条件。

b. 计算 Philly 时，其最高理论胜场为 83 场。程序构建包含 Atlanta、New York、Montreal 的比赛网络。计算发现网络发生拥堵，Atlanta 和 New York 之间 6 场比赛的胜利无论如何分配，都会导致必然有队伍的胜场超过 83，Philly 因“非平凡淘汰”失去夺冠资格。

c. 对 Atlanta 和 New York 计算最大流时，均能顺利达到满流状态，说明它们存在合理的胜负分配使得自己夺冠，故未被淘汰。

2.代码运行结果验证

将上述逻辑和数据编译运行，程序得出的判定结果与理论手工推导完全一致。这证明了将“棒球淘汰预测”转化为“流网络最大流”模型的科学性与代码实现的正确性。

运行结果截图如下：

```

4
Atlanta 83 71 8 0 1 6 1
Philly 80 79 3 1 0 0 2
New_York 78 78 6 6 0 0 0
Montreal 77 82 3 1 2 0 0

--- 赛季最终预测 ---
Atlanta: 仍有机会夺冠!
Philly: 被淘汰
New_York: 仍有机会夺冠!
Montreal: 被淘汰

D:\ccc\test\x64\Debug\test.exe (进程 4804)已退出, 代码为 0 (0x0)。
要在调试停止时自动关闭控制台, 请启用“工具”->“选项”->“调试”->“调试停止时自动关闭控制台”。
按任意键关闭此窗口。 . . .|

```

六、使用链接中的测试数据进行测试与验证

1.测试

实验选取了小规模测试集 teams5.txt 和中等规模测试集 teams12.txt 对程序进行测试, 测试的结果如下:

```

5
New_York 75 59 28 0 3 8 7 3
Baltimore 71 63 28 3 0 2 7 7
Boston 69 66 27 8 2 0 0 3
Toronto 63 72 27 7 7 0 0 3
Detroit 49 86 27 3 7 3 3 0

--- 赛季最终预测 ---
New_York: 仍有机会夺冠!
Baltimore: 仍有机会夺冠!
Boston: 仍有机会夺冠!
Toronto: 仍有机会夺冠!
Detroit: 被淘汰

D:\ccc\test\x64\Debug\test.exe (进程 33820)已退出, 代码为 0 (0x0)。
要在调试停止时自动关闭控制台, 请启用“工具”->“选项”->“调试”->“调试停止时自动关闭控制台”。
按任意键关闭此窗口。 . . .|

```

```

12
Poland 6 1 4 0 1 1 0 1 0 0 0 0 0 1 0
Russia 5 1 5 1 0 0 1 0 1 1 0 1 0 0 0
Brazil 5 1 5 1 0 0 1 0 1 0 0 1 1 0 0
Iran 5 2 4 0 1 1 0 1 0 0 0 0 0 0 1
Italy 4 2 5 1 0 0 1 0 0 1 0 1 1 0 0
Cuba 4 2 5 0 1 1 0 0 0 0 1 0 0 1 1
Argentina 3 3 5 0 1 0 0 1 0 0 1 0 0 1 1
USA 3 4 4 0 0 0 0 0 1 1 0 1 1 0 0
Japan 2 5 4 0 1 1 0 1 0 0 1 0 0 0 0
Serbia 1 5 5 0 0 1 0 1 0 0 1 0 0 1 1
Egypt 1 6 4 1 0 0 0 0 1 1 0 0 1 0 0
China 0 7 4 0 0 0 1 0 1 1 0 0 1 0 0

--- 赛季最终预测 ---
Poland: 仍有机会夺冠!
Russia: 仍有机会夺冠!
Brazil: 仍有机会夺冠!
Iran: 仍有机会夺冠!
Italy: 仍有机会夺冠!
Cuba: 仍有机会夺冠!
Argentina: 仍有机会夺冠!
USA: 仍有机会夺冠!
Japan: 被淘汰
Serbia: 被淘汰
Egypt: 被淘汰
China: 被淘汰

```

2.验证

利用 ai 工具对程序输出结果的正确性进行验证:

a.teams5.txt 验证:

Detroit 被淘汰: 完全正确。Detroit 理论最高只能达到 76 胜。而其余 4 支队伍目前的总胜场 (278) 加上它们互相之间的必须打的内战 (27 场), 总计达到 305 场。这意味着这 4 支队伍赛季结束时平均至少有 76.25 胜。根据鸽巢原理, 必然有队伍胜场大于 76, Detroit 彻底无缘冠军

b.teams12.txt (图 2) 验证:

China、Egypt 被淘汰: 完全正确。它们全赢也只有 4 胜和 5 胜, 连当前第一名 Poland 的 6 胜都达不到 (平凡淘汰)。

Japan、Serbia 被淘汰: 完全正确。它们全赢最高能拿到 6 胜。但目前排名前三的队伍 (Poland, Russia, Brazil) 加上彼此之间的内战, 最终平均每队至少能拿到 6.33 胜, 必然有队伍超越 6 胜, 所以 Japan 和 Serbia 也被淘汰 (非平凡淘汰)

七、 实验过程中的人机交互

在本实验的开展过程中, 引入了 Gemini 作为辅助编程和理论推演的工具。具体的交互过程总结如下:

1. 在初始建图完成后, 要求 AI 提供一套能够求解网络流的代码框架。AI 最初为了追求极致的时间复杂度, 直接生成了一套带有当前弧优化的高度复杂的 Dinic 算法。经过代码审查与需求评估, 我们发现: 本实验的核心诉求仅仅是判断“哪些球队有机会以最多的胜利结束赛季”, 且测试节点数量并不庞大 (最多 50 支球队级别)。Dinic 算法不仅存在过度设计的嫌疑, 而且冗长的代码极其不利于后续的查错与逻辑梳理。因此, 果断驳回了 AI 的初始方案, 并强制要求其转变思路, 改用最基础的邻接矩阵和 BFS (广度优先搜索) 来重新构建 Edmonds-Karp 算法雏形

2. AI 生成的代码往往是“自嗨型”的闭环, 缺乏对实际测试环境的适配。在拿到极简版核心代码后, 发现 AI 直接将 4 支球队的数据“硬编码”死在了程序内部, 根本无法应对 Coursera 提供的标准测试集。为此, 我们重构了 main 函数的数据读取流水线。同时设计了能够自适应截取 .txt 文件流的循环输入结构, 并为了实验报告的展示效果, 手动裁剪了 AI 原本过于啰嗦的打印提示, 将控制台输出精简为较完美的“黑盒判定”格式

3. 为验证测试数据集在我们的程序上运行后的输出结果是否正确, 我们采用了使用 AI 来计算并验证结果的方法

指导教师批阅意见：

成绩评定：

指导教师签字： 李炎然

2016 年 4 月 21 日

备注：